



深圳大学
Shenzhen University

互联网编程

第12-13章

UDP&IP组播

深圳大学计算机与软件学院

UDP

■ TCP

- 保证数据到达
- 保证数据顺序
- 速度慢
- 用途：HTTP、FTP、...



■ UDP

- 不保证一定到达
- 不保证到达顺序
- 速度快
- 用途：DNS、NFS、TFTP、...



UDP

- TCP协议
 - ServerSocket&Socket
- UDP协议
 - 没有连接（connection）概念
 - 不支持流（stream）
 - 只是数据包的发送和接收
- DatagramPacket
 - 对数据进行包装
- DatagramSocket
 - 发送和接收数据



UDP客户端

- 创建DatagramSocket对象、并设置超时

```
DatagramSocket socket = new DatagramSocket(0);
```

```
socket.setSoTimeout(10000); //10s
```

- 设置要发送DatagramPacket数据包

```
InetAddress host = InetAddress.getByName("time.nist.gov");
```

```
DatagramPacket request = new DatagramPacket(new byte[1], 1, host, 13);
```



UDP客户端

■ 创建要接收的DatagramPacket数据包

```
byte[] data = new byte[1024];  
DatagramPacket response = new DatagramPacket(data, data.length);
```

■ 发送和接收数据

```
socket.send(request);  
socket.receive(response);
```

■ 解析出数据

```
byte [] b = response.getData();
```



UDP客户端示例

DaytimeUDPClient.java

```
DaytimeUDPClient.java ✕
1 import java.io.*;
2 import java.net.*;
3 public class DaytimeUDPClient {
4     private final static int PORT = 13;
5     private static final String HOSTNAME = "time.nist.gov";
6     public static void main(String[] args) {
7         try (DatagramSocket socket = new DatagramSocket(0)) {
8             socket.setSoTimeout(10000);
9             InetAddress host = InetAddress.getByName(HOSTNAME);
10            DatagramPacket request = new DatagramPacket(new byte[1], 1, host, PORT);
11            DatagramPacket response = new DatagramPacket(new byte[1024], 1024);
12            socket.send(request);
13            socket.receive(response);
14            String result = new String(response.getData(), 0, response.getLength(),
15                                     "US-ASCII");
16            System.out.println(result);
17        } catch (IOException ex) {
18            ex.printStackTrace();
19        }
20    }
21 }
```

UDP服务器

- 实现方法跟UDP客户端类似
- 创建DatagramSocket对象

```
DatagramSocket socket = new DatagramSocket(13);
```

- 创建要接收数据的DatagramPacket对象

```
DatagramPacket request = new DatagramPacket(new byte[1024], 0, 1024);
```

- 接收数据

```
socket.receive(request);
```



UDP服务器

- 设置要发送的DatagramPacket数据包

```
byte[] data = ...;
```

```
DatagramPacket response = new DatagramPacket(data, data.length, host, port);
```

- 发送数据

```
socket.send(response);
```

UDP服务器示例DaytimeUDPServer.java



```
1 import java.net.*;
2 import java.util.Date;
3 import java.util.logging.*;
4 import java.io.*;
5 public class DaytimeUDPServer {
6     private final static int PORT = 13;
7     private final static Logger audit = Logger.getLogger("requests");
8     private final static Logger errors = Logger.getLogger("errors");
9     public static void main(String[] args) {
10         try (DatagramSocket socket = new DatagramSocket(PORT)) {
11             while (true) {
12                 try {
13                     DatagramPacket request = new DatagramPacket(new byte[1024], 1024);
14                     socket.receive(request);
15
16                     String daytime = new Date().toString();
17                     byte[] data = daytime.getBytes("US-ASCII");
18                     DatagramPacket response = new DatagramPacket(data, data.length,
19                         request.getAddress(), request.getPort());
20                     socket.send(response);
21                     audit.info(daytime + " " + request.getAddress());
22                 } catch (IOException | RuntimeException ex) {
23                     errors.log(Level.SEVERE, ex.getMessage(), ex);
24                 }
25             }
26         } catch (IOException ex) {
27             errors.log(Level.SEVERE, ex.getMessage(), ex);
28         }
29     }
30 }
```

DatagramPacket

- 接收数据包构造方法
 - `public DatagramPacket(byte[] buffer, int length)`
 - `public DatagramPacket(byte[] buffer, int offset, int length)`
- 大部分系统支持的数据包大小为8K,
- 一般不要超过8K, 虽然IPv4的理论值为65,507 (IPv6 65,536)



DatagramPacket

■ 发送数据包对象创建

```
public DatagramPacket(byte[] data, int length,  
    InetAddress destination, int port)  
public DatagramPacket(byte[] data, int offset, int length,  
    InetAddress destination, int port)  
public DatagramPacket(byte[] data, int length,  
    SocketAddress destination)  
public DatagramPacket(byte[] data, int offset, int length,  
    SocketAddress destination)
```



DatagramSocket

■ 构造方法

- `public DatagramSocket()` throws `SocketException`
- `public DatagramSocket(int port)` throws `SocketException`

```
import java.net.*;

public class UDPPortScanner {

    public static void main(String[] args) {
        for (int port = 1024; port <= 65535; port++) {
            try {
                // the next line will fail and drop into the catch block if
                // there is already a server running on port i
                DatagramSocket server = new DatagramSocket(port);
                server.close();
            } catch (SocketException ex) {
                System.out.println("There is a server on port " + port + ".");
            }
        }
    }
}
```

DatagramSocket

属性设置

- SO_TIMEOUT: 超时
- SO_RCVBUF: 接收缓存大小
- SO_SNDBUF: 发送缓存大小
- SO_REUSEADDR: 多个DatagramSocket绑定到同一个网络界面和端口
- SO_BROADCAST: 发送到广播地址和从广播地址接收
- IP_TOS: 数据包的优先级



DatagramChannel

- 功能与SocketChannel和ServerSocketChannel类似
- 以非阻塞的方式处理网络通讯



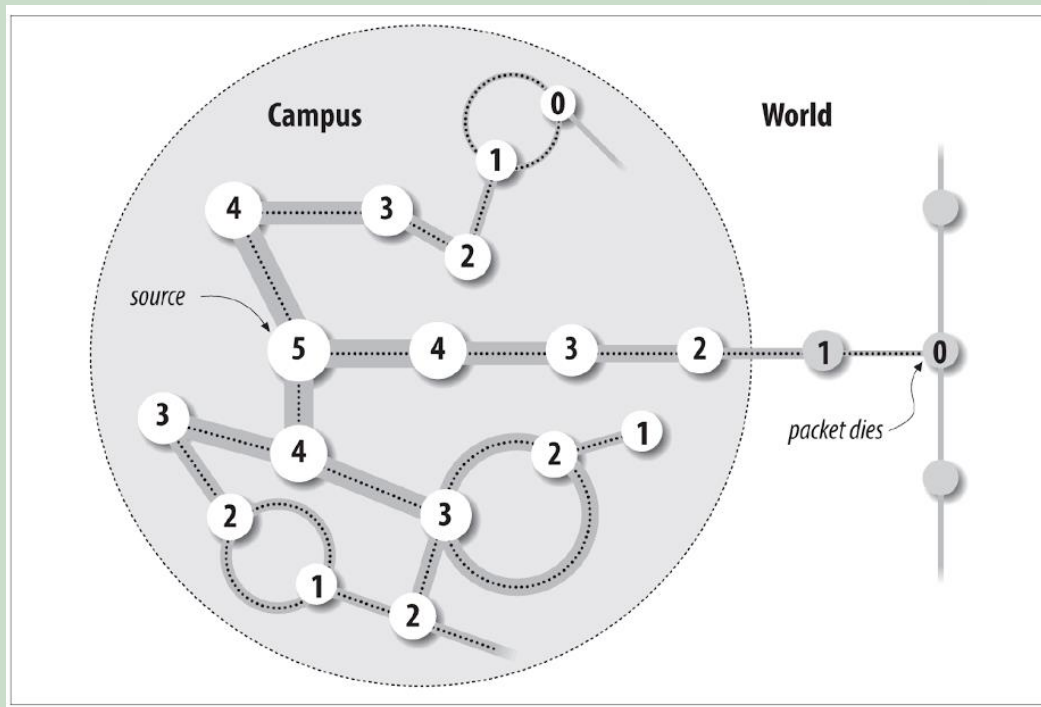
单播、广播、组播

- 单播（unicast）：点对点通信
- 广播（Broadcast）：一对所有通信
 - 网络对其中每一台主机发出的信号都进行无条件复制并转发
- 组播/多播（multicast）：一对多通信
- 组播 DNS（mDNS）工作在 IP 层面，使用5353端口。
 - 它不请求 DNS 服务器，而是在局域网内广播，
 - 所有支持组播 DNS 的设备都会回复它自己的域名，
 - 如果没有指定自身机器名或者有冲突，那么新设备就会换个名字继续广播，直到名字可用，且被其他设备所接受。



组播作用

- 用法：将数据包发送给一组地址
- 一般不应用于Internet，而用于局域网
- 使用TTL(Time To Live)限制传输距离



组播地址和组

- 组播地址（multicast address）是称为组播组(multicast group)的一组主机的共享地址。
 - IPv4组播地址: 224.0.0.0 – 239.255.255.255
 - 所有地址以1110作为前4位
- 组播组是一组共享一个组播地址的Internet主机
 - 主组地址可以是: 225.0.0.0-238.255.255.255中的任一个。
 - 任何发送给组播组地址的数据都会中继给组中的所有成员
 - 组是开放的，组中的成员主机可在任何时候加入和离开组
 - 组播组可以是临时的或永久的，永久的组播组分配的地址保持不变，不论组中是否有成员
 - 临时组播组只有在有成员时才存在



组播编程例子

- 例程MulticastSniffer实现组播窃听器用于验证确实能够接收一个特定主机的组播数据:
 - ▣ 从命令行中读取组播组名, 根据这个主机名构造 一个InetAddress,
 - ▣ 创建一个MulticastSocket, 尝试加入该主机名相应的组播组
 - ▣ 如果尝试成功, MulticastSniffer从该Socket接收数据报, 将内容显示在控制台
- 大多数组播数据是二进制的, 显示为文本时将无法理解
- 运行:
 - ▣ java MulticastSniffer 239.255.255.250 1900
 - ▣ java MulticastSniffer 238.2.2.2 1900

```
MulticastSniffer.java ✕  
1 import java.io.*;  
2 import java.net.*;  
3 public class MulticastSniffer {  
4     public static void main(String[] args) {  
5         InetAddress group = null;  
6         int port = 0;  
7         // read the address from the command line  
8         try {  
9             group = InetAddress.getByName(args[0]);  
10            port = Integer.parseInt(args[1]);  
11        } catch (ArrayIndexOutOfBoundsException | NumberFormatException  
12            | UnknownHostException ex) {
```

```
13     System.err.println(  
14         "Usage: java MulticastSniffer multicast_address port");  
15     System.exit(1);  
16 }  
17 MulticastSocket ms = null;  
18 try {  
19     ms = new MulticastSocket(port);  
20     ms.joinGroup(group);  
21     byte[] buffer = new byte[8192];  
22     while (true) {  
23         DatagramPacket dp = new DatagramPacket(buffer, buffer.length);  
24         ms.receive(dp);  
25         String s = new String(dp.getData(), "8859_1");  
26         System.out.println(s);  
27     }  
28 } catch (IOException ex) {  
29     System.err.println(ex);  
30 } finally {  
31     if (ms != null) {  
32         try {  
33             ms.leaveGroup(group);  
34             ms.close();  
35         } catch (IOException ex) {}  
36     }  
37 }  
38 }  
39 }
```

组播编程例子

- MulticastSender实现发送组播数据：
 - 从命令行读取输入（组播组的地址、端口号和一个可选的TTL），
 - 将一个字符串数据填充到字节数据data，
 - 把这个数组放在DatagramPacket dp中，
 - 构造MulticastSocket ms，加入组ia
 - ms向组ia 发送10次数据报包dp
 - TTL设置为1，确保这个数据不会传输到本地子网之外
 - 发送完数据后，ms离开组并自行关闭
- 运行：
 - java MulticastSender 238.2.2.2 1900

```
MulticastSender.java ✕
1 import java.io.*;
2 import java.net.*;
3 public class MulticastSender {
4     public static void main(String[] args) {
5         InetAddress ia = null;
6         int port = 0;
7         byte ttl = (byte) 1;
8         // read the address from the command line
```



```
9      try {
10          ia = InetAddress.getBy_name(args[0]);
11          port = Integer.parseInt(args[1]);
12          if (args.length > 2) ttl = (byte) Integer.parseInt(args[2]);
13      } catch (NumberFormatException | IndexOutOfBoundsException
14              | UnknownHostException ex) {
15          System.err.println(ex);
16          System.err.println(
17              "Usage: java MulticastSender multicast_address port ttl");
18          System.exit(1);
19      }
20      byte[] data = "Here's some multicast data\r\n".getBytes();
21      DatagramPacket dp = new DatagramPacket(data, data.length, ia, port);
22      try (MulticastSocket ms = new MulticastSocket()) {
23          ms.setTimeToLive(ttl);
24          ms.joinGroup(ia);
25          for (int i = 1; i < 10; i++) {
26              ms.send(dp);
27          }
28          ms.leaveGroup(ia);
29      } catch (SocketException ex) {
30          System.err.println(ex);
31      } catch (IOException ex) {
32          System.err.println(ex);
33      }
34  }
35 }
```

谢谢

